

Airline Database Management System

A full-stack relational database design featuring complex analytics queries, stored procedures, functions, and automated triggers for operational aviation data management.

TECH STACK MySQL · SQL · Relational Database Design · Stored Procedures · Triggers

DOMAIN Aviation Operations · Passenger Management · Analytics

Project Overview

The Airline Database Management System (ADMS) is a centralized relational database solution designed to streamline airline operations data. The system manages flights, passengers, reservations, crew assignments, aircraft, and operational analytics within a cohesive, normalized schema.

Key capabilities include role-based access control, automated reservation and scheduling processes, real-time flight status tracking, and analytics tooling that surfaces insights into passenger behavior and operational performance. The database is engineered for scalability, data integrity, and regulatory compliance.

Core Objectives → Enhance data accessibility and reliability → Streamline reservation and scheduling workflows → Centralize operational information → Enable data-driven decision-making → Improve customer service through analytics	Project Scope → Database schema design and development → Data migration from legacy systems → Booking and flight operations integration → Stored procedures and automation → Security and access control implementation
--	--

Database Schema

The database consists of 20 normalized tables spanning all major operational domains of an airline. Below is the complete Entity-Relationship diagram followed by a summary of each table.

Entity-Relationship Diagram

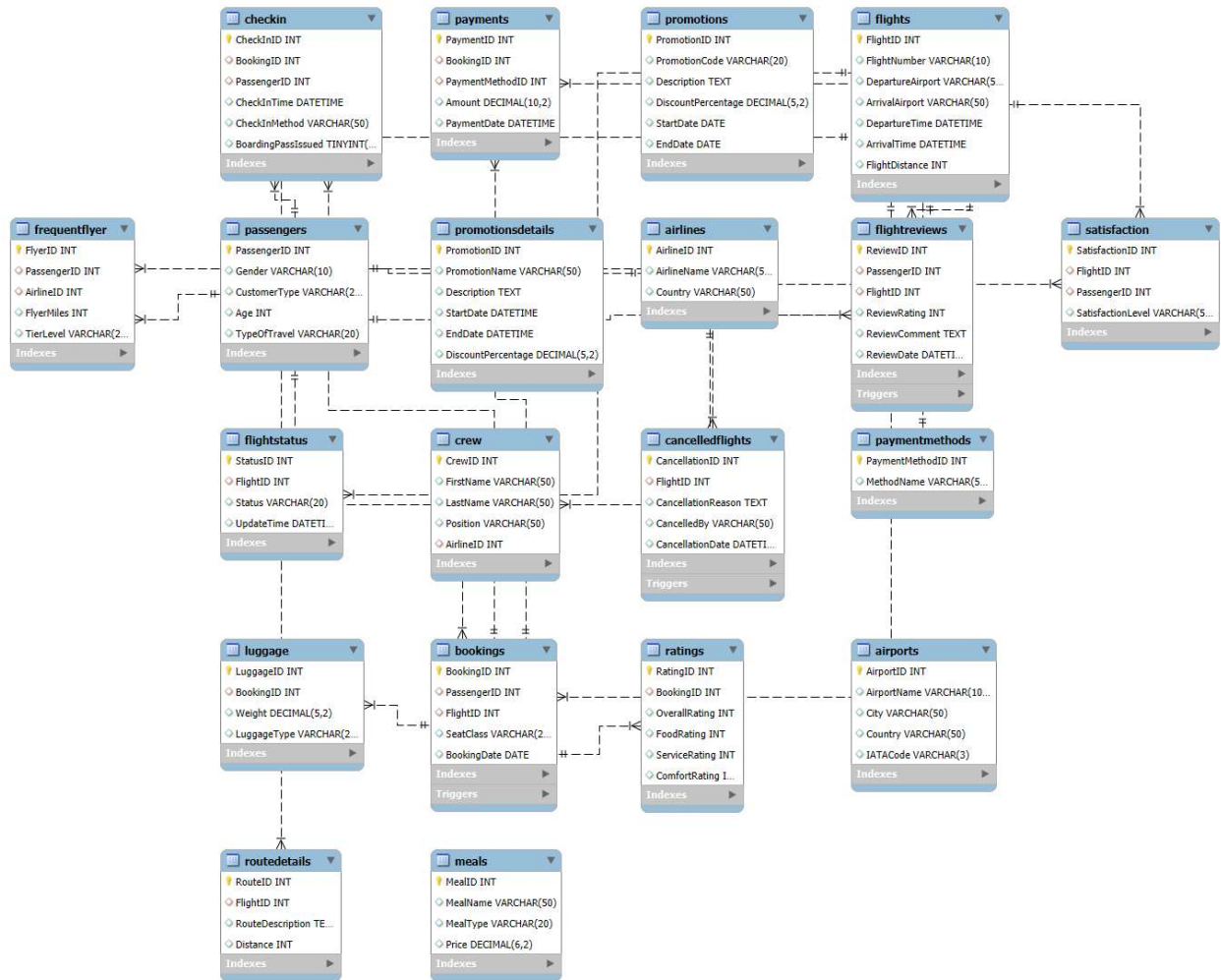


Table Inventory

Table	Description
Passengers	Passenger demographics, travel preferences, and customer type classification
Flights	Flight schedules, departure/arrival airports, and distances
Bookings	Reservation records linking passengers to flights and seat classes
Airlines	Airline company information including name and country of operation
Airports	Airport details including IATA codes and geographic location
FlightStatus	Real-time tracking of flight status (On Time, Delayed, Cancelled)
Crew	Crew member details, roles, and airline associations

Table	Description
Luggage	Passenger baggage weight, type, and tracking data
Meals	In-flight meal options and associated pricing
Ratings	Passenger ratings across food, service, and comfort dimensions
Promotions	Promotional offers with discount percentages and validity windows
PaymentMethods	Supported payment methods for bookings
Payments	Payment transaction records including amounts and dates
Satisfaction	Passenger satisfaction levels per flight
FrequentFlyer	Loyalty program enrollment, tier levels, and accumulated miles
CancelledFlights	Cancellation records with reason and responsible party
RouteDetails	Route descriptions including distance and routing metadata
FlightReviews	Passenger narrative reviews and star ratings per flight
CheckIn	Check-in records including method (online/kiosk/counter) and time
PromotionsDetails	Extended promotional offer structure with targeting metadata

Complex Analytical Queries

Ten analytical queries demonstrate multi-table joins, conditional aggregations, window functions, and subquery patterns across the database schema.

1. Top Routes by Customer Satisfaction

Ranks the top 5 flight routes by average satisfaction rate, calculated as the proportion of satisfied passengers per route.

```
SELECT f.DepartureAirport, f.ArrivalAirport,
       AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
       SatisfactionRate,
       COUNT(*) AS TotalPassengers
FROM Flights f
JOIN Satisfaction s ON f.FlightID = s.FlightID
GROUP BY f.DepartureAirport, f.ArrivalAirport
ORDER BY SatisfactionRate DESC
LIMIT 5;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	DepartureAirport	ArrivalAirport	SatisfactionRate	TotalPassengers
▶	New York	London	1.0000	1
	Los Angeles	Sydney	1.0000	1
	Chicago	Berlin	1.0000	1
	Paris	Tokyo	0.0000	1
	Dubai	Singapore	0.0000	1

2. Flight Distance and Delay Impact on Satisfaction

Examines how flight distance (short/medium/long) combined with delay status affects customer satisfaction rates.

```
SELECT
  CASE
    WHEN f.FlightDistance < 1000 THEN 'Short'
    WHEN f.FlightDistance BETWEEN 1000 AND 5000 THEN 'Medium'
    ELSE 'Long'
  END AS DistanceCategory,
  fs.Status AS DelayStatus,
  COUNT(s.SatisfactionLevel) AS TotalPassengers,
  AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) * 100 AS
  SatisfactionRate
FROM Flights f
JOIN FlightStatus fs ON f.FlightID = fs.FlightID
JOIN Satisfaction s ON f.FlightID = s.FlightID
GROUP BY DistanceCategory, DelayStatus
```

```
ORDER BY DistanceCategory, DelayStatus;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 				
	DistanceCategory	DelayStatus	TotalPassengers	SatisfactionRate
▶	Long	Cancelled	1	0.0000
	Long	Delayed	1	0.0000
	Long	On Time	3	100.0000

3. Satisfaction by Customer Type and Travel Type

Compares satisfaction rates between loyal and disloyal customers across business vs. personal travel.

```
SELECT
    p.CustomerType,
    p.TypeOfTravel,
    AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
    SatisfactionRate,
    COUNT(*) AS TotalPassengers
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
GROUP BY p.CustomerType, p.TypeOfTravel
ORDER BY p.CustomerType, p.TypeOfTravel;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 				
	CustomerType	TypeOfTravel	SatisfactionRate	TotalPassengers
▶	Disloyal Customer	Business travel	1.0000	3
	Disloyal Customer	Personal Travel	0.0000	2

4. Popular Routes by Frequent Flyer Tier

Finds the top routes for each loyalty tier, ranked by total bookings.

```
SELECT
    ff.TierLevel,
    CONCAT(f.DepartureAirport, ' to ', f.ArrivalAirport) AS Route,
    COUNT(b.BookingID) AS TotalBookings
FROM FrequentFlyer ff
JOIN Passengers p ON ff.PassengerID = p.PassengerID
JOIN Bookings b ON p.PassengerID = b.PassengerID
JOIN Flights f ON b.FlightID = f.FlightID
GROUP BY ff.TierLevel, Route
ORDER BY ff.TierLevel DESC, TotalBookings DESC
```

```
LIMIT 10;
```

Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:			
	TierLevel	Route	TotalBookings
▶	Silver	New York to London	1
	Silver	Chicago to Berlin	1
	Silver	Seoul to Moscow	1
	Platinum	Dubai to Singapore	1
	Platinum	Bangkok to San Francisco	1
	Gold	Paris to Tokyo	1
	Gold	Mumbai to Hong Kong	1
	Gold	Amsterdam to Cape Town	1
	Bronze	Los Angeles to Sydney	1
	Bronze	Toronto to São Paulo	1

5. Age Group and Seat Class Satisfaction Analysis

Examines satisfaction variation across age brackets (Under 20, 20-40, 41-60, Over 60) and seat classes.

```
SELECT
  CASE
    WHEN p.Age < 20 THEN 'Under 20'
    WHEN p.Age BETWEEN 20 AND 40 THEN '20-40'
    WHEN p.Age BETWEEN 41 AND 60 THEN '41-60'
    ELSE 'Over 60'
  END AS AgeGroup,
  b.SeatClass,
  AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
  SatisfactionRate,
  COUNT(*) AS TotalPassengers
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
JOIN Bookings b ON s.FlightID = b.FlightID
GROUP BY AgeGroup, b.SeatClass
ORDER BY AgeGroup, b.SeatClass;
```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	AgeGroup	SeatClass	SatisfactionRate	TotalPassengers
▶	20-40	Business	1.0000	2
	20-40	Economy	0.0000	2
	41-60	First Class	1.0000	1

6. Average Service Ratings by Customer Type

Calculates mean food, service, and comfort scores for each customer segment.

```
SELECT
    p.CustomerType,
    AVG(r.FoodRating) AS AvgFoodRating,
    AVG(r.ServiceRating) AS AvgServiceRating,
    AVG(r.ComfortRating) AS AvgComfortRating
FROM Passengers p
JOIN Bookings b ON p.PassengerID = b.PassengerID
JOIN Ratings r ON b.BookingID = r.BookingID
GROUP BY p.CustomerType;
```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	CustomerType	AvgFoodRating	AvgServiceRating	AvgComfortRating
▶	Disloyal Customer	3.1000	4.0000	3.6000

7. Flights with Highest Dissatisfaction Rates

Surfaces flights with the highest proportion of dissatisfied passengers, segmented by seat class and distance.

```
SELECT
    f.FlightID,
    b.SeatClass,
    f.FlightDistance,
    COUNT(*) AS TotalPassengers,
    AVG(CASE WHEN s.SatisfactionLevel = 'Neutral or Dissatisfied' THEN 1.0 ELSE 0
END) AS DissatisfactionRate
FROM Flights f
JOIN Satisfaction s ON f.FlightID = s.FlightID
JOIN Bookings b ON f.FlightID = b.FlightID
GROUP BY f.FlightID, b.SeatClass, f.FlightDistance
ORDER BY DissatisfactionRate DESC
LIMIT 10;
```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	FlightID	SeatClass	FlightDistance	TotalPassengers	DissatisfactionRate
▶	2	Economy	9713	1	1.00000
	4	Economy	5841	1	1.00000
	1	Business	5567	1	0.00000
	3	First Class	12051	1	0.00000
	5	Business	6856	1	0.00000

8. Distance Category vs. Satisfaction by Customer Type

Cross-tabulates customer type with flight distance category to reveal satisfaction patterns.

```
SELECT
    p.CustomerType,
    CASE
        WHEN f.FlightDistance < 500 THEN 'Short'
        WHEN f.FlightDistance BETWEEN 500 AND 2000 THEN 'Medium'
        ELSE 'Long'
    END AS FlightDistanceCategory,
    AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
    SatisfactionRate
FROM Passengers p
JOIN Satisfaction s ON p.PassengerID = s.PassengerID
JOIN Flights f ON s.FlightID = f.FlightID
GROUP BY p.CustomerType, FlightDistanceCategory
ORDER BY p.CustomerType, FlightDistanceCategory;
```

Result Grid		Filter Rows:		Export:		Wrap Cell Content:	
	CustomerType	FlightDistanceCategory	SatisfactionRate				
▶	Disloyal Customer	Long	0.6000				

9. Luggage Weight by Seat Class and Flight

Aggregates total baggage weight per flight and seat class for operational planning.

```
SELECT
    f.FlightNumber,
    b.SeatClass,
    SUM(l.Weight) AS TotalLuggageWeight
FROM Flights f
JOIN Bookings b ON f.FlightID = b.FlightID
JOIN Luggage l ON b.BookingID = l.BookingID
GROUP BY f.FlightNumber, b.SeatClass
ORDER BY f.FlightNumber, TotalLuggageWeight DESC;
```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	FlightNumber	SeatClass	TotalLuggageWeight			
▶	FL001	Business	23.50			
	FL002	Economy	18.70			
	FL003	First Class	30.00			
	FL004	Economy	15.20			
	FL005	Business	22.80			
	FL006	Economy	10.50			
	FL007	Business	28.30			
	FL008	Economy	17.90			
	FL009	First Class	35.00			
	FL010	Economy	12.40			

10. Most Common Low-Rated Service Combinations

Identifies frequent co-occurrence of low ratings (< 3) across food, service, and comfort by meal type.

```

SELECT
    m.MealName AS FoodType,
    CASE WHEN r.FoodRating < 3 THEN 1 ELSE 0 END AS LowFoodRating,
    CASE WHEN r.ServiceRating < 3 THEN 1 ELSE 0 END AS LowServiceRating,
    CASE WHEN r.ComfortRating < 3 THEN 1 ELSE 0 END AS LowComfortRating,
    COUNT(*) AS Occurrences
FROM Ratings r
JOIN Bookings b ON r.BookingID = b.BookingID
JOIN Meals m ON b.BookingID = m.MealID
GROUP BY FoodType, LowFoodRating, LowServiceRating, LowComfortRating
ORDER BY Occurrences DESC
LIMIT 10;

```

Result Grid						Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	FoodType	LowFoodRating	LowServiceRating	LowComfortRating	Occurrences				
▶	Chicken Pasta	0	0	0	1				
	Vegetarian Curry	1	0	0	1				
	Beef Steak	0	0	0	1				
	Vegan Salad	1	0	1	1				
	Seafood Platter	0	0	0	1				
	Gluten-free Sandwich	0	0	0	1				
	Kosher Meal	0	0	0	1				
	Halal Lamb	1	0	0	1				
	Fruit Platter	0	0	0	1				
	Kids Meal	1	0	1	1				

Stored Procedures

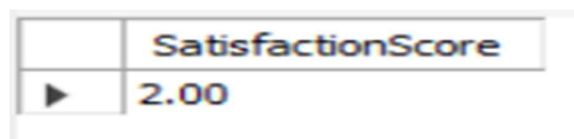
Five stored procedures encapsulate reusable business logic, reducing application-layer complexity and ensuring consistent enforcement of business rules at the database level.

1. CalculateFlightSatisfaction

Computes an aggregated satisfaction score for a specific flight by averaging food, service, comfort, and overall ratings.

```
DELIMITER //
CREATE PROCEDURE CalculateFlightSatisfaction(IN flightID INT, OUT satisfactionScore
DECIMAL(4,2))
BEGIN
    SELECT AVG(
        r.FoodRating + r.ServiceRating + r.ComfortRating +
        IFNULL(r.OverallRating, 0)
    ) / 4 INTO satisfactionScore
    FROM Ratings r
    JOIN Bookings b ON r.BookingID = b.BookingID
    WHERE b.FlightID = flightID;
END //
DELIMITER ;

-- Usage
SET @satisfactionScore = 0;
CALL CalculateFlightSatisfaction(4, @satisfactionScore);
SELECT @satisfactionScore AS SatisfactionScore;
```



SatisfactionScore
2.00

2. GetPassengerSatisfactionHistory

Retrieves full satisfaction history for a given passenger including flight details, seat class, and satisfaction level ordered by most recent.

```
DELIMITER //
CREATE PROCEDURE GetPassengerSatisfactionHistory(IN passengerID INT)
BEGIN
    SELECT f.FlightID, b.SeatClass, s.SatisfactionLevel, f.DepartureTime
    FROM Flights f
    JOIN Satisfaction s ON f.FlightID = s.FlightID
    JOIN Bookings b ON s.PassengerID = b.PassengerID
    WHERE s.PassengerID = passengerID
    ORDER BY f.DepartureTime DESC;
```

```

END //
DELIMITER ;

-- Usage
CALL GetPassengerSatisfactionHistory(3);

```

	FlightID	SeatClass	SatisfactionLevel	DepartureTime
►	3	First Class	Satisfied	2024-11-03 23:15:00

3. GenerateSatisfactionReport

Produces a satisfaction metrics report for flights within a specified date range, broken down by seat class.

```

DELIMITER //
CREATE PROCEDURE GenerateSatisfactionReport(IN startDate DATE, IN endDate DATE)
BEGIN
    SELECT
        b.SeatClass,
        COUNT(*) AS TotalPassengers,
        AVG(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 1 ELSE 0 END) AS
        SatisfactionRate,
        AVG(r.FoodRating) AS AvgFoodRating,
        AVG(r.ServiceRating) AS AvgServiceRating,
        AVG(r.ComfortRating) AS AvgComfortRating
    FROM Flights f
    JOIN Satisfaction s ON f.FlightID = s.FlightID
    JOIN Bookings b ON f.FlightID = b.FlightID
    JOIN Ratings r ON b.BookingID = r.BookingID
    WHERE DATE(f.DepartureTime) BETWEEN startDate AND endDate
    GROUP BY b.SeatClass
    ORDER BY SatisfactionRate DESC;
END //
DELIMITER ;

```

	SeatClass	TotalPassengers	SatisfactionRate	AvgFoodRating	AvgServiceRating	AvgComfortRating
►	Business	2	1.0000	3.5000	4.5000	4.0000
	First Class	1	1.0000	5.0000	5.0000	5.0000
	Economy	2	0.0000	1.5000	3.5000	2.5000

4. IdentifyFrequentComplainers

Flags passengers whose dissatisfaction count exceeds a configurable threshold, returning complaint counts and average ratings.

```

DELIMITER //
CREATE PROCEDURE IdentifyFrequentComplainers(IN complaintThreshold INT)
BEGIN
    SELECT
        p.PassengerID,
        p.CustomerType,
        COUNT(*) AS TotalFlights,
        SUM(CASE WHEN s.SatisfactionLevel = 'Neutral or Dissatisfied' THEN 1 ELSE 0
END) AS Complaints,
        AVG(r.FoodRating + r.ServiceRating + r.ComfortRating +
IFNULL(r.OverallRating, 0)) / 4 AS AvgRating
    FROM Passengers p
    JOIN Satisfaction s ON p.PassengerID = s.PassengerID
    JOIN Bookings b ON s.FlightID = b.FlightID AND s.PassengerID = b.PassengerID
    JOIN Ratings r ON b.BookingID = r.BookingID
    GROUP BY p.PassengerID, p.CustomerType
    HAVING Complaints >= complaintThreshold
    ORDER BY Complaints DESC;
END //
DELIMITER ;

-- Usage
CALL IdentifyFrequentComplainers(1);

```

	PassengerID	CustomerType	TotalFlights	Complaints	AvgRating
▶	2	Disloyal Customer	1	1	3.00000000
	4	Loyal Customer	1	1	2.00000000

5. UpdateCustomerType

Batch-updates passenger customer type classification based on total flight count: passengers with more than 10 bookings are labelled Loyal Customer.

```

DELIMITER //
CREATE PROCEDURE UpdateCustomerType()
BEGIN
    UPDATE Passengers p
    JOIN (
        SELECT PassengerID, COUNT(*) AS FlightCount
        FROM Bookings
        GROUP BY PassengerID
    ) b ON p.PassengerID = b.PassengerID
    SET p.CustomerType =
        CASE
            WHEN b.FlightCount > 10 THEN 'Loyal Customer'
            ELSE 'Disloyal Customer'
        END;
END;

```

```
END //
DELIMITER ;

-- Usage
SELECT PassengerID, CustomerType FROM Passengers;
```

	PassengerID	CustomerType
▶	1	Loyal Customer
	2	Disloyal Customer
	3	Loyal Customer
	4	Loyal Customer
	5	Disloyal Customer

Stored Functions

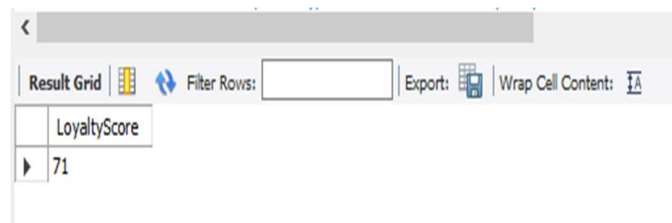
Five deterministic functions implement reusable calculations that can be called inline within queries, enabling composable analytics logic.

1. CalculateCustomerLoyaltyScore

Computes a composite loyalty score for a passenger based on flight count, satisfaction history, and total distance flown.

```
DELIMITER //
CREATE FUNCTION CalculateCustomerLoyaltyScore(passengerID INT)
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE loyaltyScore INT;
    SELECT
        COUNT(*) * 10 +
        SUM(CASE WHEN s.SatisfactionLevel = 'Satisfied' THEN 5 ELSE 0 END) +
        SUM(f.FlightDistance / 100) INTO loyaltyScore
    FROM Satisfaction s
    JOIN Flights f ON s.FlightID = f.FlightID
    WHERE s.PassengerID = passengerID;
    RETURN COALESCE(loyaltyScore, 0);
END //
DELIMITER ;

-- Usage
SELECT CalculateCustomerLoyaltyScore(1) AS LoyaltyScore;
```



LoyaltyScore
71

2. DeterminePassengerTier

Maps a numeric loyalty score to a tier label: Bronze, Silver, Gold, or Platinum.

```
DELIMITER //
CREATE FUNCTION DeterminePassengerTier(loyaltyScore INT)
RETURNS VARCHAR(20) DETERMINISTIC
BEGIN
    DECLARE tier VARCHAR(20);
    IF loyaltyScore > 1000 THEN SET tier = 'Platinum';
    ELSEIF loyaltyScore > 500 THEN SET tier = 'Gold';
    ELSEIF loyaltyScore > 200 THEN SET tier = 'Silver';
```

```

    ELSE SET tier = 'Bronze';
  END IF;
  RETURN tier;
END //
DELIMITER ;

-- Usage
SELECT DeterminePassengerTier(72) AS PassengerTier;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PassengerTier			
Bronze			

3. CalculateFlightPunctuality

Returns the punctuality percentage for a given flight ID based on on-time status records in FlightStatus.

```

DELIMITER //
CREATE FUNCTION CalculateFlightPunctuality(flightID INT)
RETURNS DECIMAL(5,2) DETERMINISTIC
BEGIN
  DECLARE totalFlights INT DEFAULT 0;
  DECLARE onTimeFlights INT DEFAULT 0;
  SELECT COUNT(*) INTO totalFlights FROM FlightStatus WHERE FlightID = flightID;
  SELECT COUNT(*) INTO onTimeFlights FROM FlightStatus
    WHERE FlightID = flightID AND Status = 'On Time';
  RETURN (onTimeFlights * 100.0) / GREATEST(totalFlights, 1);
END //
DELIMITER ;

-- Usage
SELECT CalculateFlightPunctuality(2) AS PunctualityPercentage;

```

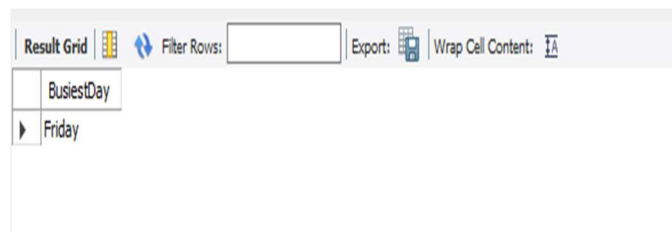
Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PunctualityPercentage			
60.00			

4. GetBusiestDayOfWeek

Returns the day-of-week name with the highest flight volume, useful for scheduling and staffing planning.

```
DELIMITER //
CREATE FUNCTION GetBusiestDayOfWeek()
RETURNS VARCHAR(10) DETERMINISTIC
BEGIN
    DECLARE busiestDay VARCHAR(10);
    SELECT CASE DayOfWeek
        WHEN 1 THEN 'Sunday' WHEN 2 THEN 'Monday' WHEN 3 THEN 'Tuesday'
        WHEN 4 THEN 'Wednesday' WHEN 5 THEN 'Thursday'
        WHEN 6 THEN 'Friday' WHEN 7 THEN 'Saturday'
    END AS DayOfWeekName INTO busiestDay
    FROM (
        SELECT DAYOFWEEK(DepartureTime) AS DayOfWeek, COUNT(*) AS FlightCount
        FROM Flights
        GROUP BY DAYOFWEEK(DepartureTime)
        ORDER BY FlightCount DESC LIMIT 1
    ) AS SubQuery;
    RETURN busiestDay;
END //
DELIMITER ;

-- Usage
SELECT GetBusiestDayOfWeek() AS BusiestDay;
```



The screenshot shows a database query result grid. At the top, there is a toolbar with buttons for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below the toolbar, the result is displayed in a table with one column labeled 'BusiestDay' and one row containing the value 'Friday'.

BusiestDay
Friday

Database Triggers

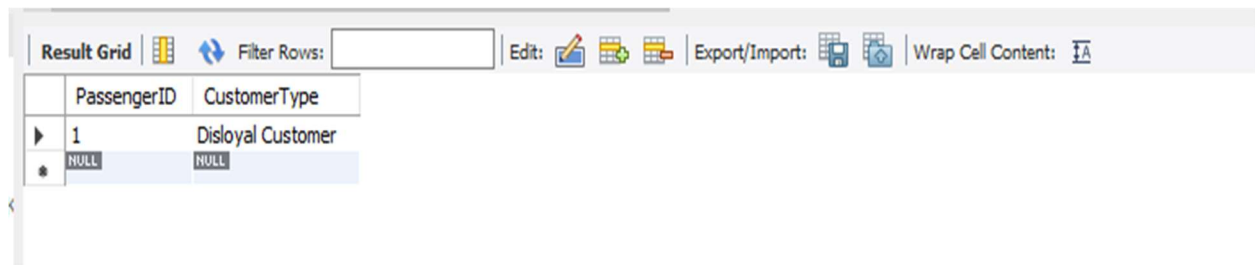
Five triggers enforce business rules and maintain data integrity automatically at the row-level, without requiring application-layer enforcement.

1. UpdateCustomerTypeAfterBooking

AFTER INSERT on Bookings. Automatically reclassifies passenger customer type to Loyal Customer once they exceed 10 total bookings.

```
DELIMITER //
CREATE TRIGGER UpdateCustomerTypeAfterBooking
AFTER INSERT ON Bookings FOR EACH ROW
BEGIN
    DECLARE flight_count INT;
    SELECT COUNT(*) INTO flight_count FROM Bookings WHERE PassengerID =
NEW.PassengerID;
    IF flight_count > 10 THEN
        UPDATE Passengers SET CustomerType = 'Loyal Customer'
        WHERE PassengerID = NEW.PassengerID;
    ELSE
        UPDATE Passengers SET CustomerType = 'Disloyal Customer'
        WHERE PassengerID = NEW.PassengerID;
    END IF;
END //
DELIMITER ;

-- Test
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (1, 2, 'Economy', '2024-12-01');
SELECT PassengerID, CustomerType FROM Passengers WHERE PassengerID = 1;
```



	PassengerID	CustomerType
▶	1	Disloyal Customer
•	NULL	NULL

2. AddCancellationToReviews

AFTER INSERT on CancelledFlights. Automatically creates a 1-star review entry in FlightReviews with the cancellation reason.

```
DELIMITER //
CREATE TRIGGER AddCancellationToReviews
AFTER INSERT ON CancelledFlights FOR EACH ROW
```

```

BEGIN
    INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment,
    ReviewDate)
        VALUES (NULL, NEW.FlightID, 1,
            CONCAT('Flight cancelled due to: ', NEW.CancellationReason),
            NEW.CancellationDate);
END //
DELIMITER ;

-- Test
INSERT INTO CancelledFlights (FlightID, CancellationReason, CancelledBy,
CancellationDate)
VALUES (3, 'Technical issues', 'Maintenance', '2024-12-01 10:00:00');
SELECT * FROM FlightReviews WHERE FlightID = 3;

```

ReviewID	PassengerID	FlightID	ReviewRating	ReviewComment	ReviewDate
3	3	3	3	Average flight, could be better.	2024-11-03 06:00:00
11	NULL	3	1	Flight cancelled due to: Technical issues	2024-12-01 10:00:00
NULL	NULL	NULL	NULL	NULL	NULL

3. PreventBookingOnCancelledFlights

BEFORE INSERT on Bookings. Raises a SQLSTATE 45000 error if a passenger attempts to book a flight that has been cancelled.

```

DELIMITER //
CREATE TRIGGER PreventBookingOnCancelledFlights
BEFORE INSERT ON Bookings FOR EACH ROW
BEGIN
    DECLARE is_cancelled INT;
    SELECT COUNT(*) INTO is_cancelled FROM CancelledFlights WHERE FlightID =
    NEW.FlightID;
    IF is_cancelled > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking on a cancelled flight is not allowed.';
    END IF;
END //
DELIMITER ;

-- Test: expected error
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (2, 3, 'Business', '2024-12-02');

```

✖ 30 15:56:53 INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate) VALUES (2, 3, 'Busi... Error Code: 1644. Booking on a cancelled flight is not allowed.

4. PreventDuplicateFlightReview

BEFORE INSERT on FlightReviews. Prevents a passenger from submitting more than one review per flight.

```
DELIMITER //
CREATE TRIGGER PreventDuplicateFlightReview
BEFORE INSERT ON FlightReviews FOR EACH ROW
BEGIN
    DECLARE review_count INT;
    SELECT COUNT(*) INTO review_count FROM FlightReviews
        WHERE PassengerID = NEW.PassengerID AND FlightID = NEW.FlightID;
    IF review_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Duplicate review not allowed for the same passenger and
flight.';
    END IF;
END //
DELIMITER ;

-- Test
INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment,
ReviewDate)
VALUES (1, 2, 5, 'Excellent flight!', '2024-12-01 14:00:00');
-- Second insert below should return error:
INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment,
ReviewDate)
VALUES (1, 2, 4, 'Good flight overall.', '2024-12-02');
```

 32 15:58:17 INSERT INTO FlightReviews (PassengerID, FlightID, ReviewRating, ReviewComment, ReviewDate) VALUES (1, 2, 4, 'Good flight overall.', '2024-12-02') Error Code: 1644. Duplicate review not allowed for the same passenger and flight.

5. UpdateFrequentTravelerStatus

AFTER INSERT on Bookings. Reclassifies a passenger as Frequent Traveler once their cumulative bookings reach 10.

```
DELIMITER //
CREATE TRIGGER UpdateFrequentTravelerStatus
AFTER INSERT ON Bookings FOR EACH ROW
BEGIN
    DECLARE booking_count INT;
    SELECT COUNT(*) INTO booking_count FROM Bookings WHERE PassengerID =
NEW.PassengerID;
    IF booking_count >= 10 THEN
        UPDATE Passengers SET CustomerType = 'Frequent Traveler'
        WHERE PassengerID = NEW.PassengerID;
    END IF;
END //
DELIMITER ;
```

```
-- Test
INSERT INTO Bookings (PassengerID, FlightID, SeatClass, BookingDate)
VALUES (1, 6, 'Economy', '2024-12-02');
SELECT PassengerID, CustomerType FROM Passengers WHERE PassengerID = 3;
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
PassengerID	CustomerType			
1	Disloyal Customer			
NULL	NULL			

Conclusion

The Airline Database Management System delivers a robust, scalable relational database architecture purpose-built for modern aviation operations. By integrating passenger management, flight scheduling, booking workflows, crew assignment, and operational analytics into a cohesive schema, the system eliminates data silos and automates key business processes.

The modular design ensures adaptability as passenger volumes grow and operational requirements evolve. Stored procedures and functions abstract complex logic into reusable, testable units. Triggers enforce referential integrity and business rules at the database layer, reducing reliance on application-level validation.

Advanced analytics queries expose actionable patterns in customer satisfaction, loyalty program engagement, and operational performance — providing a foundation for data-driven strategy. The system positions airlines to operate with greater efficiency, stronger data governance, and an improved passenger experience.

20 Database Tables	10 Complex Queries	5 Stored Procedures	5 Triggers
--	--	---	----------------------------------